# Maximal triangle sum

**Programming assignment 1**
Algorithms and Complexity

## 1 Problem Description

We consider pyramids of numbers where on the first line there is one number, on the second line two numbers, on the third line three numbers, and so on. Design a program that finds a path in this triangle, from the top line to the bottom line, such that the sum of the numbers in this path is maximized. The next number in a path lies either directly below to the left of the current chosen number or directly below to the right of the current chosen number.
Consider for example the following triangle:

```
   3
  7 5
 1 4 5
9 5 8 2
```

The bolded path in this triangle is the path with the maximum sum.

## 2 Input/Output

The numbers in the pyramid are strictly between 0 and 100 and are padded with extra zeros at the beginning if necessary. The output expected is the sum of the maximum path.

### 2.1 Example Input

```
03
07 05
01 04 05
09 05 08 02
```

### 2.2 Example Output

```
22
```

## 3 Submission

**Note:** This programming assignment is largely auto-graded. So make sure to adhere exactly to the guidelines below, otherwise your code cannot be graded.

#### Check your code before submission

By executing `check.sh` in the same directory as your Python and input/output files, you can verify that your code produces the correct output on the small input (`small.input`) and has the correct input format.

#### Grading Criteria

- The code must be written in Python 3.
- The code reads input from *stdin* and writes to *stdout*. To easily test input, you can use input redirection:
  `python ex1.py < small.input`

  *Hint: The following example code reads from stdin and writes the same to stdout correctly.*

```
import fileinput
for line in fileinput.input():
    # line already contains a newline character
    print(line, end="")
```

- The code file itself must be named `ex1.py`.
- Submit **only** the Python file with your code and name it `ex1.py`. Do not submit a zip/tar file or anything similar.

#### How Will the Code Be Graded?

Your code will be evaluated based on two input files:

- Provided is `small.input` with the corresponding output `small.output`. If your code produces the same output with `small.input` as input, a score of 6 will be assigned.

*Note: To achieve this score, it is necessary that you have reasonably implemented the algorithm. Simply returning the same output every time will not be considered correct.*

- Additionally, your code will be checked on a large input, of same size (but different content) as the 'large.input' file that was supplied. If your code also produces the correct output for this input, a score of 10 will be assigned.
  *Note: In particular, your code should be efficient enough to run on the large input – timing out counts as failing. In general, for all exercises a solution exists which does not take a lot of run time.*

Besides this automated check, the code will also be checked for plagiarism from others and code on the internet. The general plagiarism and fraud regulations of the UvA apply.

**Other Questions**

If you have other questions/problems regarding the input format, ask a question during the lab session or post your question on Canvas / via email.